

Módulo 12: Principios SOLID aplicados en Python

Contenidos clave

SRP, OCP, LSP, ISP, DIP en clave “pythonic”:

- SRP: Single Responsibility Principle - principio de Responsabilidad Única: Una clase debería tener una sola responsabilidad principal.
- OCP: Open Closed Principle - Abierto para extensión, pero cerrado para modificación.
- LSP: Liskov Substitution Principle: principio de sustitución de Liskov: Una implementación debe poder sustituir a otra sin romper el comportamiento esperado.
- ISP: Interface Segregation Principle - principio de segregación de interfaces: Es mejor tener interfaces pequeñas y específicas que una interfaz enorme.
- DIP - principio de inversión de dependencias: Los módulos de alto nivel no deben depender directamente de implementaciones concretas. Deben depender de abstracciones.

Inversión de dependencias con Protocols, factories y provider patterns:

- Protocol permite definir formalmente qué comportamiento se espera.
- Factory: se encarga de crear la implementación que necesitamos.
- Provider Pattern: un Provider proporciona una dependencia al componente que lo necesita.

Acoplamiento, cohesión y testabilidad:

- Acoplamiento: indica cuánto depende un componente de otro.
- Cohesión: indica qué tan relacionadas están las responsabilidades dentro de un componente.
- Testabilidad: SOLID mejora la testabilidad.

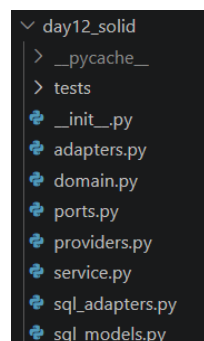
Objetivos

- **Aplicar SOLID a servicios y dominio.** Se separó el sistema en componentes con responsabilidades claras:

- Dominio: entidad de Order en [domain.py](#)
- Puerto: contrato OrderRepository definido con Protocol.
- Servicio: OrderService, encargado de los casos de uso.
- Adaptadores: persistencia en memoria y con SQLAlchemy.
- Provider: construcción e inyección de dependencias.

Aplicando:

1. SRP: cada módulo tiene una responsabilidad específica.
2. DIP: el servicio depende del protocolo, no de una implementación concreta.



```
day12_solid
├── __pycache__
├── tests
├── __init__.py
├── adapters.py
├── domain.py
├── ports.py
├── providers.py
├── service.py
├── sql_adapters.py
└── sql_models.py
```

3. OCP: es posible agregar nuevos repositorios sin modificar OrderService.
4. ISP: el protocolo expone únicamente save y get_by_id.
5. LSP: los repositorios pueden sustituirse sin cambiar el comportamiento esperado.

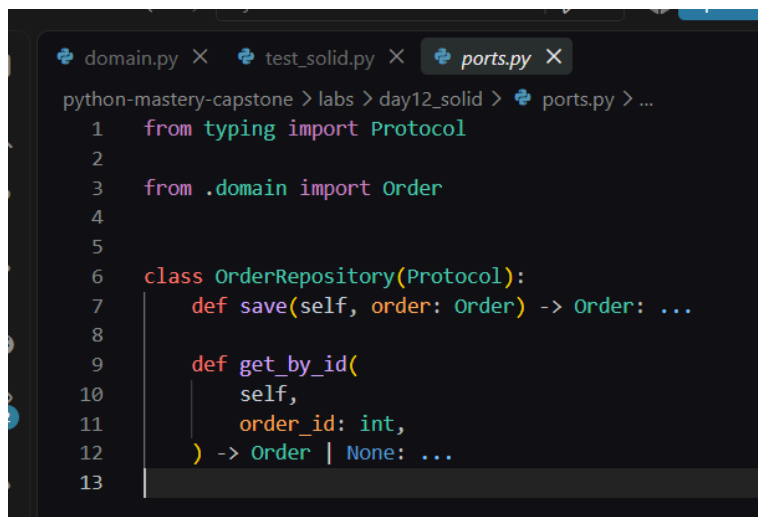
- Reducir acoplamiento y mejorar extensibilidad.

OrderService recibe un OrderRepository mediante inyección de dependencias, por tanto, el servicio no conoce si los datos se guardan en: un diccionario en memoria, en SQLite mediante SQLAlchemy u otra implementación futura; lo anterior facilita cambiar la persistencia, probar el servicio con repositorios simples, incorporar nuevos adaptadores y mantener el dominio independiente de la infraestructura.

Laboratorio

- Refactor de servicio para depender de puerto (Protocol).

Se tiene el servicio independiente de un puerto, se definió el siguiente contrato:



```
python-mastery-capstone > labs > day12_solid > ports.py > ...
1  from typing import Protocol
2
3  from .domain import Order
4
5
6  class OrderRepository(Protocol):
7      def save(self, order: Order) -> Order: ...
8
9      def get_by_id(
10         self,
11         order_id: int,
12     ) -> Order | None: ...
13
```

OrderService depende de este protocolo y no de clases concretas.

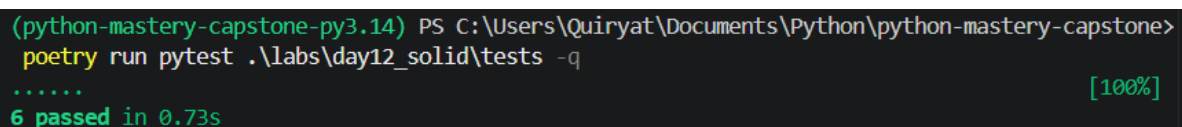
- implementaciones memoria/SQL y verificación LSP.

Se creó InMemoryOrderRepository, para pruebas rápidas, desarrollo local y validar la lógica del servicio sin base de datos.

Se creó SQLAlchemyOrderRepository, que utiliza SQLAlchemy y SQLite para persistir las órdenes mediante un modelo de infraestructura. Ambos adaptadores exponen la misma interfaz al servicio.

Se utilizaron pruebas parametrizadas para ejecutar el mismo conjunto de operaciones con:

- InMemoryOrderRepository
- SQLAlchemyOrderRepository



```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
poetry run pytest .\labs\day12_solid\tests -q
..... [100%]
6 passed in 0.73s
```

Katherine Quiryat Rosales Garduño

Ambas implementaciones pudieron sustituirse sin modificar OrderService, confirmando el cumplimiento de LSP. Se validó la implementación SQL usando una base SQLite temporal, evitando afectar datos reales.